BABEŞ–BOLYAI TUDOMÁNYEGYETEM KOLOZSVÁR
MATEMATIKA ÉS INFORMATIKA KAR
INFORMATIKA SZAK

Szakdolgozat

Távoli hitelesítési szolgáltatás beágyazott rendszerekhez az ARM TrustZone-M alapjain



TÉMAVEZETŐ:

DR. LIBÁL ANDRÁS,
DOCENS

SZERZŐ:

NAGY CSONGOR

2026

BABEŞ–BOLYAI UNIVERSITY OF CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND INFORMATICS
SPECIALIZATION: COMPUTER SCIENCE

Diploma Thesis

Remote Attestation Service for Embedded Systems Based on ARM TrustZone-M



ADVISOR:

ANDRÁS LIBÁL, PHD.
ASSOCIATE PROFESSOR

AUTHOR:

CSONGOR NAGY

2026

UNIVERSITATEA BABEŞ–BOLYAI, CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ

Lucrare de licență

Serviciu de atestare la distanță pentru sisteme încorporate bazate pe ARM TrustZone-M



CONDUCĂTOR ŞTIINŢIFIC:

DR. ANDRÁS LIBÁL,
CONFERENȚIAR

ABSOLVENT:

CSONGOR NAGY

2026

Abstract (EN)

This thesis presents the design and implementation of a remote attestation service for embedded systems based on ARM TrustZone-M technology running on an STM32U5 microcontroller. The attestation mechanism enables an external verifier to assess the firmware integrity of a resource-constrained IoT device, guaranteeing that the device runs an untampered and trusted software image.

The system leverages the hardware-enforced separation provided by TrustZone-M, which partitions the processor into a Secure World (Secure Enclave) and a Non-Secure World. The Secure World implements the core attestation engine, computing a cryptographic SHA-256 measurement of the Non-Secure firmware directly from memory. This measurement is performed entirely within the Secure Enclave, ensuring that potentially compromised Non-Secure software cannot falsify the attestation result.

To provide versatile security profiles that balance maximum cryptographic assurance with fast, energy-efficient execution, the architecture supports a dual-mode authentication pipeline: a low-latency symmetric HMAC-SHA256 path and a highly secure asymmetric ECDSA-SHA256 path using the SECP256R1 elliptic curve. Asymmetric operations and secure random number generation are handled via the integration of the Mbed TLS cryptographic library inside the Secure Enclave.

Communication between the isolated microcontroller and the remote verifier is bridged via an external network proxy (an ESP32 proxy). The remote Flask-based backend validates the received attestation response using either a shared symmetric key or a pre-provisioned public key.

Performance benchmarks reflect the realistic computational costs associated with embedded cryptographic operations. The symmetric HMAC attestation path executes with a latency of 1.75 to 1.9 seconds, optimized directly by the hardware hash peripheral. The asymmetric ECDSA signing path, heavily reliant on complex big-integer modular arithmetic on the elliptic curve, completes within 3.1 to 3.5 seconds. The proposed architecture successfully demonstrates how modern microcontrollers with TrustZone-M support can be paired with software cryptographic libraries to build practical, hardware-backed security foundations for decentralized IoT ecosystems.

This work is the result of my own activity. I have never provided or received unauthorized assistance in this work.

Tartalomjegyzék

1. Bevezetés **1**

1.1. Motiváció . . . 1

1.2. A dolgozat céljai . . . 2

2. Alapok **4**

2.1. Hitelesítési modellek . . . 4

2.1.1. A RATS keretrendszer . . . 4

2.1.2. Hitelesítés típusai . . . 4

2.1.3. Bizalmi lánc (Chain of Trust) . . . 5

2.2. Kriptográfiai alapok . . . 5

2.2.1. SHA256 hash függvény . . . 5

2.2.2. HMAC-SHA256 . . . 6

2.2.3. ECDSA (Elliptic Curve Digital Signature Algorithm) . . . 7

2.2.4. Nonce alapú replay támadás elleni védelem . . . 7

2.3. Megbízható végrehajtási környezet (TEE) . . . 8

2.3.1. Alternatív TEE technológiák . . . 8

2.3.2. Az ARM TrustZone-M szerepe . . . 9

2.3.3. Kapcsolódó és korábbi munkák . . . 11

3. Gyakorlati megvalósítás **14**

3.1. Rendszer architektúra és elméleti terv . . . 14

3.2. Fejlesztési környezet . . . 14

3.2.1. Alkalmazott technológiák és fejlesztői eszközök . . . 14

3.2.2. Projekt struktúra . . . 15

3.2.3. TrustZone-M konfiguráció . . . 16

3.3. Secure World implementáció . . . 16

3.3.1. Mbed TLS konfiguráció és beágyazott optimalizálás . . . 16

3.3.2. A hitelesítési motor architektúrája . . . 17

3.3.3. A hitelesítési jelentés és az aláírás generálása . . . 18

3.4. Non-Secure World-beli kommunikáció . . . 20

3.5. Hálózati kommunikáció és ESP32 proxy . . . 20

3.5.1. DoS védelem és Rate Limiting . . . 21

3.6. Python szerver oldal . . . 21

3.6.1. Hitelesítő modul . . . 21

3.6.2. SQLite adatbázis séma 23
3.6.3. Felhasználói felület 24
4. Eredmények értékelése és konklúzió **28**
4.1. Implementációs tapasztalatok 28
4.1.1. TrustZone-M projekt konfiguráció 28
4.1.2. NSC veneer pointer biztonság 29
4.1.3. Hardver akcelerátor megszakíthatósága 29
4.2. Teljesítményi adatok összegzése 29
4.2.1. Biztonsági értékelés 31
4.2.2. Mérési módszertan 32
4.3. Biztonsági konklúziók 35
4.3.1. Elért biztonsági garanciák 35
4.3.2. Ismert korlátok 35
4.4. Kiterjesztési lehetőségek 36
4.4.1. Over-the-Air (OTA) firmware frissítés 36
4.4.2. Többeszközös hitelesítés és fleet kezelés 36
4.4.3. Alternatív hálózati protokollok: MQTT és CoAP 37
4.4.4. Hordozhatóság más mikrokontrollerekre 37
Összefoglaló **39**

1. fejezet

Bevezetés

1.1. Motiváció

Az elmúlt évtizedben az internetre csatlakoztatott beágyazott rendszerek száma robbanásszerűen megnőtt. Az iparági előrejelzések szerint 2034-re több mint 40 milliárd IoT eszköz lesz üzemben világszerte [17]. Ezek az eszközök kritikus infrastruktúrában, egészségügyi rendszerekben, ipari automatizálásban és otthoni automatizációban egyaránt megjelennek. Ezek olyan területek, ahol a biztonság megsértése súlyos, akár életveszélyes következményekkel is járhat.

Az IoT eszközök biztonsági kihívásai több forrásból erednek. Egyrészt ezek az eszközök erőforrásaikban erősen korlátozottak: tipikusan pár megabájt Flash memóriával és néhány száz kilobájt RAM-mal rendelkeznek, amelyek nem teszik lehetővé a hagyományos, nehézsúlyú biztonsági megoldások (például a teljes TLS verem vagy dedikált TPM chipek) közvetlen alkalmazását. Másrészt az IoT eszközök fizikailag hozzáférhető, sokszor felügyelet nélküli helyszíneken működnek, ahol a hardveres és lokális támadások reális fenyegetést jelentenek. Harmadrészt a firmware-frissítések kezelése nehézkes, így a sérülékeny verziók potenciálisan hosszú ideig maradnak kiszolgáltatott állapotban.

Ezeket a gyengeségeket használják ki a modern, fejlett kártevők is, amelyek egyre gyakrabban törekednek a perzisztencia (tartós jelenlét) elérésére. Míg a korai IoT malware-ek (mint a klasszikus Mirai) többnyire csak RAM-ban léteztek és egy újraindítással letörölhetők voltak, a modern fenyegetések már képesek tartósan megtelepedni a Flash memóriában vagy a bootloader szintjén.

A fentiek alapján fő kérdésünk az, hogy miként valósítható meg ezen eszközök megbízható, futásidejű integritás ellenőrzése anélkül, hogy az drasztikusan növelné az IoT eszközök hardveres költségeit. Emellett fontos, hogy a bevezetett védelem milyen mértékű számítási és teljesítménybeli többletterhelést (*overhead*) jelent, és hogyan integrálható a firmware-be úgy, hogy az ne akadályozza a mikrokontroller alapvető feladatainak ellátását.

1.2. A dolgozat céljai

A dolgozat fő célja egy olyan komplett, biztonságos és erőforrásban hatékony távoli hitelesítési (*Remote Attestation*) protokoll és architektúra kifejlesztése, amely az STM32U5 mikrokontroller hardveres TrustZone-M képességeire épít. A célok elérése érdekében a projekt a firmware hitelesítésétől kezdve a hálózati kommunikáción át egészen a szerveroldali kiértékelésig terjedő zárt folyamatot valósít meg.

A dolgozat felépítése

A dolgozat hátralevő része a következőképpen van strukturálva:

- A 2. fejezet bemutatja az elméleti hátteret, részletesen ismertetve az ARM TrustZone-M architektúra alapelveit, a memória particionálását és a biztonsági átmeneteket. Ugyanitt ismertetésre kerülnek a felhasznált kriptográfiai alapok (SHA256, HMAC-SHA256, ECDSA), a RATS hitelesítési keretrendszer, Nonce alapú támadások, valamint az STM32U5 mikrokontroller hardveres sajátosságai.
- A 3. fejezet a gyakorlati megvalósítást tárgyalja lépésről lépésre, bemutatva a projekt struktúráját, a *Secure World*-ben futó hitelesítési motort, a *Non-Secure Callable* (NSC) kapukat, a *Non-Secure* oldali HTTP szervert, valamint a Python alapú verifikációs backend és a SQLite adatbázis megvalósítását, illetve ennek a felhasználói felületét.
- A 4. fejezet összegzi az implementációs tapasztalatokat, bemutatja a teljesítménybeli eredményeket, elemzi az elért biztonsági garanciákat, valamint részletezi a rendszer jövőbeli kiterjesztési lehetőségeit (mint amilyen az OTA frissítés vagy a hálózati protokollok lecserélése MQTT-re).

Köszönetnyilvánítás és a projekt életciklusa

Ezúton szeretném kifejezni hálás köszönetemet mentoromnak, **Dr. Libál Andrásnak**, aki szakmai iránymutatásával támogatta dolgozatom elkészítését.

A jelen dokumentumban bemutatott munka egy hosszabb fejlesztési és kísérletezési folyamat eredménye. A projekt bemutatásra került az Erdélyi Tudományos Diákköri Konferencián (ETDK), ahol a szakmai zsűri értékelése alapján **második helyezést** ért el. Az ott kapott építő jellegű visszajelzések nagyban hozzájárultak a rendszer finomításához és a dolgozat végleges formájának kialakításához.

2. fejezet

Alapok

2.1. Hitelesítési modellek

2.1.1. A RATS keretrendszer

Az IETF Remote ATtestation procedureS (RATS) [13] munkacsoport egységes terminológiát definiál a távoli hitelesítés számára. A RATS három fő szerepkört azonosít:

1. **Attester**: Az eszköz, amelynek integritását igazolni kell (STM32U5).
2. **Verifier**: Az entitás, amely kiértékeli a bizonyítékokat (Python szerver).
3. **Relying Party**: Az entitás, amely a hitelesítési eredmény alapján dönt a bizalomról.

2.1.2. Hitelesítés típusai

– **Boot-time Attestation / IAS**: Az integritás ellenőrzése a rendszerindítás során történik. A boot folyamat minden kritikus komponensét (bootloader, firmware, operációs rendszer) kriptográfiai méréseknek vetik alá. Az attesztáció a rendszer indításkori állapotát igazolja, azonban nem képes kimutatni azokat a támadásokat, amelyek a sikeres bootolást követően történnek.

– **On-Demand Attestation**: A rendszer állapotának mérése futásidőben történik, egy izolált és megbízható végrehajtási környezet segítségével. A hitelesítési folyamat nem korlátozódik a boot idejére, hanem a rendszer aktuális állapotáról szolgáltat bizonyítékot. Ez lehetővé teszi a futás közben bekövetkező kompromittálódások észlelését is.

– **Control-Flow Attestation (CFA)**: Nem csupán a flash memória vagy a firmware integritását ellenőrzi, hanem a program tényleges végrehajtási útvonalát is. A rendszer rögzíti

a fontos vezérlésátadási eseményeket (például függvényhívásokat, visszatéréseket és ugrásokat). A módszer alkalmas olyan támadások detektálására is, amelyek nem módosítják a programkódot, hanem annak végrehajtási sorrendjét manipulálják, például Return-Oriented Programming (ROP) vagy Control-Flow Hijacking támadások esetén.

2.1.3. Bizalmi lánc (Chain of Trust)

A jelen rendszerben a bizalmi lánc a TrustZone Secure World területén tárolt Secure firmware-ből indul ki. A Secure firmware tartalmazza az attesztációs kulcsot és a hitelesítési logikát, valamint ellenőrzi a Non-Secure firmware integritását. Mivel a Secure World memóriaterülete hardveresen védett, a Non-Secure alkalmazás kompromittálódása esetén sem módosítható az attesztációs mechanizmus vagy a titkos kulcs. Ez biztosítja, hogy a távoli hitelesítés eredménye megbízhatóan tükrözze a Non-Secure firmware aktuális állapotát.

Verifier (Python szerver)
↑ HMAC-SHA256 aláírás ellenőrzése
Hálózati kommunikáció (HTTP/HTTPS)
↑ Hitelesítési válasz
Non-Secure World (hálózati réteg)
↑ IPC hívás
Secure World (hitelesítési motor)
↑ Hardver-szintű elkülönítés
STM32U5 TrustZone-M hardver

2.1. ábra. A bizalmi lánc rétegei a hitelesítési rendszerben

2.2. Kriptográfiai alapok

2.2.1. SHA256 hash függvény

A SHA256 (*Secure Hash Algorithm 256*) az NSA által tervezett SHA-2 hash függvény család tagja [11]. Tetszőleges hosszúságú üzenetből 256 bites (32 bájtos) hash értéket (üzenetlenyomatot) számít, amely az alábbi biztonsági tulajdonságokkal bír:

1. **Preimage-rezisztencia**: Adott h hash értékből számítási szempontból nem lehetséges megtalálni azt az m üzenetet, amelyre $\text{SHA256}(m) = h$.

2. **Második preimage-rezisztencia**: Adott m_1 üzenetből számítási szempontból nem lehetséges megtalálni azt az $m_2 \neq m_1$ üzenetet, amelyre SHA256$(m_1) =$ SHA256(m_2).

3. **Ütközésrezisztencia**: Számítási szempontból nem lehetséges megtalálni két különböző $m_1 \neq m_2$ üzenetet, amelyekre SHA256$(m_1) =$ SHA256(m_2).

Az algoritmus 64 körből áll, és minden körben 8 darab 32 bites állapotszót (a, b, c, d, e, f, g, h) frissít.

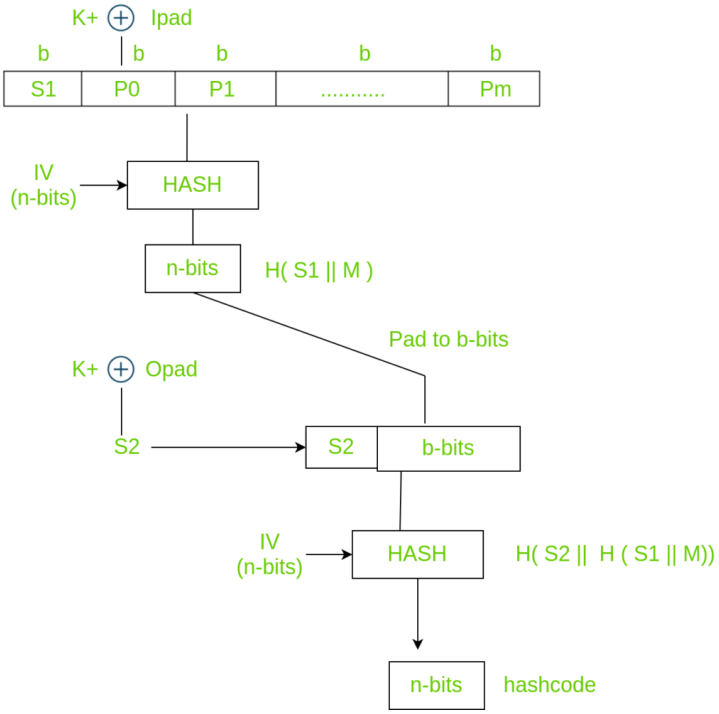
2.2.2. HMAC-SHA256

A HMAC (*Hash-based Message Authentication Code*) [6] egy kriptográfiai üzenethitelesítési kód, amely egy hash függvényt és egy titkos kulcsot kombinál.

A HMAC-SHA256 képlete:

$$\text{HMAC}(K, M) = \text{SHA256}\big((K \oplus \text{opad}) \| \text{SHA256}((K \oplus \text{ipad}) \| M)\big) \tag{2.1}$$

A két különböző pad érték (ipad és opad) biztosítja, hogy a belső és külső hash számítások különböző kulcsokkal dolgozzanak, megakadályozva a hosszabbítási támadásokat.



2.2. ábra. HMAC-SHA256 ábrázolása

2.2.3. ECDSA (Elliptic Curve Digital Signature Algorithm)

Az Elliptikus Görbe Digitális Aláírási Algoritmus (ECDSA) [18] egy aszimmetrikus kriptográfiai eljárás, amely az elliptikus görbék matematikáján alapul [9]. A hagyományos RSA algoritmussal szemben az ECDSA sokkal rövidebb kulcshosszal képes ugyanazt a biztonsági szintet nyújtani, ami különösen előnyös erőforrás-korlátozott beágyazott rendszereken, például az STM32U5 mikrokontroller esetében.

Az elliptikus görbék általános Weierstrass egyenlete egy véges test felett a következőképpen írható fel:

$$y^2 = x^3 + ax + b$$

A távoli hitelesítési rendszerben az ECDSA használata a következőképpen történik a validálási folyamat során:

1. **Kulcspár elosztása**: Az eszköz Secure World memóriája tartalmaz egy aszimmetrikus privát kulcsot (d), amely szigorúan védett és onnan nem olvasható ki[cite: 102]. A Verifier (Python szerver) rendelkezik a hozzá tartozó publikus kulccsal ($Q = d \times G$, ahol G a görbe egy nyilvános generátor pontja)[cite: 103].

2. **Aláírás (Signing)**: Amikor a mikrokontroller kiszámítja a firmware memóriaképének SHA256 hash értékét, a Secure World az ECDSA algoritmussal és a védett privát kulccsal digitálisan aláírja ezt az üzenetet (beleértve a nonce értéket is a replay támadások elkerülése végett).

3. **Validálás (Verification)**: A szerveroldali hitelesítő a beérkező hitelesítési választ a megfelelő publikus kulcs segítségével ellenőrzi. Ha a matematikai ellenőrzés sikeres, az egyértelműen bizonyítja, hogy az üzenet a megfelelő privát kulccsal rendelkező, kompromittálatlan eszköztől származik, és a mérés közben nem módosították.

Az ECDSA validáció bevezetése a szimmetrikus HMAC eljáráshoz képest robusztusabb, aszimmetrikus modellt biztosít. Ennek fő előnye, hogy a verifikátornak nem kell ismernie a titkos kulcsot a hitelesítéshez (csak a publikus felet), így a szerver esetleges kompromittálódása esetén sem kerül veszélybe az IoT eszközök privát kulcsa.

2.2.4. Nonce alapú replay támadás elleni védelem

A replay (visszajátszásos) támadás során egy támadó lementi egy korábbi, érvényes hitelesítési választ, és azt egy másik időpontban visszajátssza a verifiernek. A hitelesítési rendszerek

elleni replay támadások ellen két fő mechanizmus véd:

1. **Időbélyeg**: A hitelesítési aláírás tartalmazza a kérés időpontját. A verifier elutasít minden olyan választ, amelynek időbélyege 120 másodpercnél régebbi.
2. **Nonce**: A verifier minden kéréshez egyedi véletlenszerű értéket generál, amelyet az eszköznek bele kell foglalnia az aláírásba.

2.3. Megbízható végrehajtási környezet (TEE)

A megbízható végrehajtási környezet (TEE – *Trusted Execution Environment*) egy olyan izolált környezet, amely a fő operációs rendszertől függetlenül fut, biztosítva a benne tárolt adatok integritását és bizalmasságát. Az ARM TrustZone-M mellett számos más architektúra és szoftveres keretrendszer létezik, amelyek eltérő módon közelítik meg az izoláció kérdését.

2.3.1. Alternatív TEE technológiák

Intel SGX (Software Guard Extensions)

Az Intel SGX az x86-os architektúra kiterjesztése, amely lehetővé teszi az alkalmazások számára, hogy úgynevezett *enklávékat* (enclaves) hozzanak létre. Az izoláció itt nem rendszerszintű, hanem folyamatszintű: a memória egy része titkosított, és még a privilegizált szoftverek (pl. operációs rendszer, hipervizor) sem férhetnek hozzá az enklávé tartalmához. Ez a megközelítés különösen a felhőalapú számítástechnikában és szerverkörnyezetekben terjedt el.

ARM TrustZone (Cortex-A)

Bár a név azonos, a nagy teljesítményű Cortex-A sorozatú processzorok (pl. okostelefonok, beágyazott Linux rendszerek) TrustZone megvalósítása jelentősen eltér a TrustZone-M-től. Míg a TrustZone-M memóriacím alapú partícionálást használ a gyors és determinisztikus környezetváltás érdekében, a Cortex-A processzorok egy különleges *Monitor* módot és egy hardveres állapotot (NS-bit) alkalmaznak. Ez lehetővé teszi egy teljes, különálló biztonságos operációs rendszer párhuzamos futtatását a normál operációs rendszer mellett.

OP-TEE (Open Portable TEE)

A TrustZone hardveres alapjaira épülve alakultak ki az azt kihasználó szoftveres megoldások, amelyek közül a legelterjedtebb a Linaro által karbantartott OP-TEE. Az OP-TEE egy nyílt

forráskódú biztonságos operációs rendszer.

Architektúrájában a *Normal World* (Rich Execution Environment) tipikusan egy gazda operációs rendszert (például Linuxot) futtat, amely egy kliens alkalmazáson keresztül kommunikál a biztonságos oldallal. A *Secure World* futtatja az OP-TEE OS-t, amely elszigetelt, úgynevezett megbízható alkalmazásokat (*Trusted Applications* – TA) kezel. Ez a megközelítés éles kontrasztban áll az erőforrásban korlátozott TrustZone-M rendszerekkel (mint amilyen a jelen dolgozatban bemutatott STM32U5 implementáció), ahol nincs szükség egy nehézsúlyú operációs rendszerre, és az izoláció magán a firmware-en belül történik.

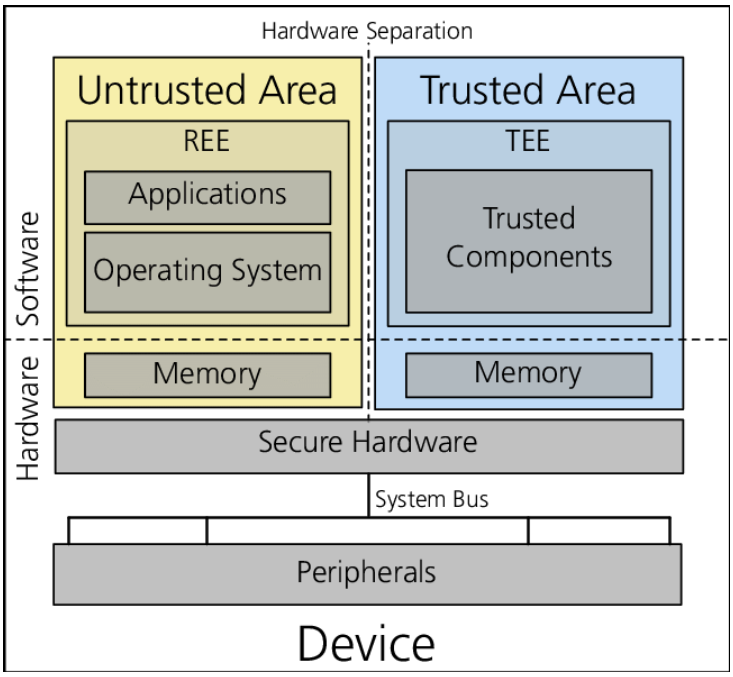
RISC-V Keystone

A nyílt forráskódú RISC-V architektúra esetében a Keystone keretrendszer kínál TEE megoldást. Ez a technológia a RISC-V hardveres memóriavédelmi funkcióit (*Physical Memory Protection* – PMP) és egy biztonságos monitort használ az elszigetelt végrehajtási környezetek létrehozásához. Legfőbb előnye a teljes átláthatóság, mivel mind a szoftveres, mind a hardveres tervek (RTL kód) nyílt forráskódúak.

2.3.2. Az ARM TrustZone-M szerepe

Az ARM TrustZone-M technológia egy TEE (Trusted Execution Environment), amely az ARM Cortex-M processzorcsalád biztonsági kiterjesztése, melyet a Cortex-M23 és Cortex-M33 processzorokban vezettek be az ARMv8-M architektúra részeként [1]. A technológia hardver szintű izolációt biztosít a processzor két egymástól elkülönített világa, a *Secure World* és a *Non-Secure World* között. Ez az izoláció a processzor utasításkészlet szintjén érvényesül: a Non-Secure kód hardver által kényszerített módon nem férhet hozzá a Secure World memóriájához és perifériáihoz.

A TrustZone-M alapelve, hogy a processzor minden hozzáférési kísérlethez (memóriaolvasás, memóriaírás, periféria-hozzáférés, kivételkezelés) egy *biztonsági attribútumot* rendel: `Secure` (S) vagy `Non-Secure` (NS). Ezt az attribútumot a hardver automatikusan propagálja: ha a vezérlés a Secure kódban van, minden implicit memória hozzáférés Secure attribútummal rendelkezik. A *Security Attribution Unit* (SAU) és a *Implementation Defined Attribution Unit* (IDAU) határozzák meg, hogy egy adott fizikai cím melyik biztonsági tartományba esik.



2.3. ábra. TEE architektúrák felépítése

Memória felosztása

A TrustZone-M-ben a memória tartományok statikusan konfigurálhatók. Az STM32U5 mikrokontrollerben az alábbi particionálás lehetséges:

Memória	**Méret**	**Hozzáférhetőség**
Secure Flash	1MB	Csak Secure World
Non-Secure Flash	1MB	Non-Secure World
Secure SRAM1	786 KB	Csak Secure World
Non-Secure SRAM2	786 KB	Non-Secure World
Non-Secure Callable (NSC)	8 KB	Átmeneti kapu

2.1. táblázat. STM32U5 memória konfigurációja TrustZone-M esetén

A *Non-Secure Callable* (NSC) tartomány különösen fontos: ez az a memóriaterület, ahol a Secure World által meghatározott függvények belépési pontjai (veneer-ek) helyezkednek el. A Non-Secure kód kizárólag ezen a kapun keresztül hívhat Secure függvényeket, ami biztosítja, hogy az átmenet ellenőrzött és auditálható legyen.

Biztonsági átmenetek

A két világ közötti vezérlésátadás a `SG` (Secure Gateway) utasítással valósul meg, amelyet kizárólag az NSC régióban elhelyezett veneer kód tartalmazhat. Amikor a Non-Secure kód meghív egy Secure függvényt, a processzor automatikusan Secure módba vált, elmenti a Non-Secure kontextust (stack pointer, visszatérési cím), és átadja a vezérlést a Secure kódnak. Visszatéréskor a `BXNS` utasítás gondoskodik a Non-Secure módra való visszaváltásról és a kontextus helyreállításáról.

Fontos biztonsági tulajdonság, hogy az átmenetek során a Secure World regiszterei *nem* kerülnek átadásra a Non-Secure kódnak: a processzor automatikusan nullázza a Secure kontextusban lévő regisztereket visszatérés előtt, megakadályozva az érzékeny adatok szivárgását.

Perifériák biztonsági konfigurációja

Az STM32U575 mikrokontrollerben a perifériákhoz is rendelhető biztonsági attribútum a *System Security Controller* (GTZC) segítségével. A Secure World számára fenntartott perifériák:

- **HASH processzor**: A hardveresen gyorsított SHA256 számításhoz.
- **RNG** (Random Number Generator): Kriptográfiai véletlenszám-generáláshoz.

2.3.3. Kapcsolódó és korábbi munkák

A megbízható végrehajtási környezetek és a távoli hitelesítési protokollok kutatása az IoT hálózatok exponenciális növekedésével párhuzamosan vált a kiberbiztonság egyik legintenzívebb területévé. Pinto és Santos [12] munkája átfogó elemzést nyújt a TrustZone és TrustZone-M architektúrákról, rámutatva a technológia alacsony szintű szoftveres izolációs lehetőségeire. A következőkben részletesen áttekintjük azokat a meghatározó tudományos munkákat és iparági szabványokat, amelyek a jelen dolgozatban bemutatott architektúra elméleti és gyakorlati előzményeit képezik.

Hibrid és minimalista attesztációs architektúrák

Az elosztott hálózatok és az erőforrásban rendkívül korlátozott eszközök elterjedésével a kutatások egy jelentős része a hibrid (hardver-szoftver együttes tervezésű) távoli hitelesítés felé fordult. Egyik legkorábbi és legmeghatározóbb munkája ezen a területen az El Defrawy és

szerzőtársai által bemutatott **SMART (Secure and Minimal Architecture for Establishing a Dynamic Root of Trust)** [4]. A SMART olyan alacsony teljesítményű mikrokontrollereket (például AVR, MSP430) céloz meg, amelyek nem rendelkeznek hardveres memóriavédelmi egységgel (MPU) vagy dedikált TEE-vel. Ehelyett a memória busz logikájának egy minimális hardveres módosítását alkalmazzák.

Egy későbbi munka a **HYDRA (HYbrid Design for Remote Attestation)** [5]. A HYDRA a tisztán szoftveres/hardveres hibrid megoldások közötti szakadékot egy formálisan ellenőrzött mikrokernel (mint a seL4) bevezetésével hidalja át. A SMART egyik fő korlátja, hogy a hitelesítési mérés megszakíthatatlan, így potenciálisan hosszú időre blokkolja az IoT eszközt. A HYDRA a mikrokernel feladatütemezésének segítségével képes megbízhatóan izolálni a hitelesítési taszkot, biztosítva annak lefutását még egy kompromittált operációs rendszer mellett is. Ugyanakkor egy ilyen mikrokernel futtatása jelentős memóriatöbblettel és feldolgozási idővel jár.

TEE-alapú IoT hitelesítés és Rootkit detektálás

A megbízható végrehajtási környezetek elterjedése lehetővé tette a kiterjesztett firmware és operációs rendszer szintű attesztációs eljárások kidolgozását. Ezen a területen alapozó jellegű Shepherd és mások [15] munkája, amely bemutatja, hogyan építhető ki kölcsönösen megbízható kommunikációs csatorna (mutually trusted channel) távoli szenzoros IoT eszközök és az ellenőrző entitások között, kifejezetten a TEE izolációs képességeire támaszkodva. Kiemelik, hogy a hardveresen elválasztott környezetből indított kriptográfiai igazolások kulcsfontosságúak a hamisított mérési adatok kiszűrésében.

Szorosan kapcsolódik ehhez az elvhez a BME CrySyS Laboratórium kutatóinak munkája, a **T-RAID (TEE-based Remote Attestation for IoT Devices)** rendszer [8]. A T-RAID egy IoT eszközökre fejlesztett attesztációs keretrendszer, amelynek elsődleges célja a hálózati rootkit-ek detektálása. Egy egyedi attesztációs lekérdező nyelvet (attestation query language) vezetnek be, amellyel a szerver a TEE belsejéből futtatott célzott méréseket indíthat: ellenőrizheti a hálózati verem (network stack) kernelfüggvény mutatóit, valamint a kritikus fájlrendszeri útvonalak és rendszerhívási táblák (syscall tables) integritását.

A T-RAID architektúra jellemzően összetettebb operációs rendszert futtató platformokra (például Cortex-A sorozat) optimalizált, ahol magasabb absztrakciós szinten értelmezhető a fájlrendszer és a kernel adatstruktúráinak kompromittálódása. A jelen dolgozat által bemutatott megoldás ezzel szemben szigorúan erőforrás-korlátozott, "bare-metal" Cortex-M33 mikrokontrollerekre koncentrál. Ebben az esetben az integritás mérése nem operációs rendszeri struk-

túrákra, hanem közvetlenül a Non-Secure firmware Flash memóriájának statikus kriptográfiai hash-elésére épül. Bár az ellenőrzött elemek eltérőek, az alapelv megegyezik: a mérés és a válasz kriptográfiai aláírása mindkét esetben egy hardveresen elzárt környezetben, a TEE alatt történik.

Iparági szabványok és referencia architektúrák

Az egyedi kutatási architektúrák mellett elengedhetetlen a rendszer elhelyezése a modern iparági standardok között. Az ARM által kidolgozott PSA Certified (Platform Security Architecture) [3] modell egységes útmutatást biztosít az IoT platformok biztonságos hardver- és firmware-tervezéséhez. Ennek a keretrendszernek a gyakorlati megvalósulása a Trusted Firmware-M (TF-M) [2], amely az ARMv8-M specifikáció nyílt forráskódú referenciája. A TF-M a Secure World-ben futva biztosít megbízható szolgáltatásokat, mint amilyen a Biztonságos Tárolás (Secure Storage), a védett Kriptográfiai modul, és az Initial Attestation Service.

A TF-M attesztációs válaszai szorosan illeszkednek a RATS (Remote ATtestation procedureS) modellhez. Ezen belül az adatok hordozhatóságát és standardizálását az IETF által kidolgozott Entity Attestation Token (EAT) [7] biztosítja, amely CBOR adatformátumra és COSE kriptográfiai üzenetekre támaszkodva foglalja szabványba a hitelesítési "claim"-eket.

A jelen dolgozat elhelyezése

A jelen dolgozat mindezen munkáktól abban különbözik, hogy bár épít a korábbi modellek elméleti alapjaira, elkerüli a TF-M robusztus, általános célú absztrakciós rétegeinek bevezetését. A TF-M keretrendszer integrálása és kódmérete inkább a komplex, nagyméretű ipari projektekhez készült, míg ez a munka közvetlenül az **ARM TrustZone-M** architektúra natív hardveres izolációs képességeire támaszkodik egy alacsony szinten implementált, dedikált Secure World firmware segítségével.

Emellett a projekt kifejezett célja a futásidejű hitelesítés megvalósítása. Szemben a hagyományos, kizárólag a rendszerindításkor lefutó statikus ellenőrzésekkel (Secure Boot / SRTA), ez a TrustZone-alapú architektúra lehetővé teszi a Non-Secure firmware integritásának igény szerinti (*on-demand*), a működés bármely időpillanatában történő ellenőrzését.

3. fejezet

Gyakorlati megvalósítás

3.1. Rendszer architektúra és elméleti terv

A hitelesítési rendszer háromrétegű architektúrán alapul:

1. **Beágyazott eszköz**: Secure World (hitelesítési motor) és Non-Secure World (hálózat I/O).
2. **Hálózati szállítás**: HTTP/HTTPS alapú csatorna a payload továbbítására.
3. **Szerver oldal**: Python verifikációs réteg és relációs adatbázis az ellenőrzéshez.

3.2. Fejlesztési környezet

3.2.1. Alkalmazott technológiák és fejlesztői eszközök

Mikrokontroller (STM32U5) réteg

- **STM32CubeIDE**: Az STMicroelectronics Eclipse alapú integrált fejlesztői környezete a beágyazott C kód írásához és hibakereséséhez
- **STM32CubeMX**: Grafikus kódgeneráló és konfigurációs eszköz, amely a TrustZone-M memóriafelosztás és a perifériák (pl. hardveres HASH) beállítására szolgál.
- **STM32CubeProgrammer**: Az STMicroelectronics programozó és memóriakezelő szoftvere, amely a lefordított firmware mikrokontrollerre történő flashelésére, különböző opciók beállítására (pl. TZEN byte), valamint a memóriatartalom ellenőrzésére és menedzselésére szolgál.
- **Mbed TLS 4.1.0**: Nyílt forráskódú, erőforrás-korlátozott beágyazott rendszerekre optimalizált kriptográfiai könyvtár, amely a *Secure World* környezetben végzett aszimmetrikus kulcskezelési és digitális aláírási (ECDSA) műveletek szoftveres hátterét biztosítja.

Hálózati kommunikációs réteg (ESP32)

- **C/C++ (Arduino Core)**: Az ESP32 hálózati modul programozásához használt környezet.
- **WiFi.h és WebServer.h**: A vezeték nélküli hálózati csatlakozást és a HTTP-alapú kommunikációt biztosító beépített könyvtárak, amelyek a payload továbbításáért és a REST API-val való kapcsolattartásért felelnek.

Szerveroldali architektúra (Backend)

- **Python 3.11 & Flask**: A szerveroldali hitelesítési logika (HMAC/ECDSA verifikáció) és a REST API végpontok megvalósításáért felelős keretrendszer.
- **SQLite**: Relációs adatbáziskezelő (RDBMS) a hitelesítési eredmények, eszközadatok és kriptográfiai kulcsok strukturált naplózásához.

Vizualizációs réteg (Frontend)

- **React & Vite**: A felhasználói felület (dashboard) alapja. A Vite modern, gyors build eszközként szolgál.
- **TypeScript**: Típusos programozási nyelv, amely csökkenti a futásidejű hibákat és robusztusabbá teszi a kliensoldali kódot.
- **Material UI (MUI)**: Kész, testreszabható felületi komponenseket biztosít.
- **Axios**: Promise based HTTP kliens a backend API-val történő aszinkron adatcseréhez és a naplózás lekérdezéséhez.

3.2.2. Projekt struktúra

A firmware kétprojektes felépítést alkalmaz, amely az STM32CubeIDE TrustZone-M munkafolyamatának megfelelő:

```
attestation-system/
├── firmware/
│   ├── Secure/
│   └── NonSecure/
└── server/
    └── Backend/
```

```
    app.py
    verifier.py
    database.py
  Frontend/
esp32_proxy/
```

3.2.3. TrustZone-M konfiguráció

A TrustZone-M memória felosztása az STM32CubeMX segítségével konfigurálható. A konfiguráció lényegi eleme a `GTZC` (Global TrustZone Controller) beállítása, amely a Flash és SRAM tartományok biztonsági attribútumait rögzíti, ezt az STM32Programmer segítségével lehet konfigurálni.

3.3. Secure World implementáció

3.3.1. Mbed TLS konfiguráció és beágyazott optimalizálás

Az aszimmetrikus kriptográfiai primitívek (ECDSA) megvalósításához az Mbed TLS könyvtárat integráltuk a mikrokontroller Secure World környezetébe. Mivel az Mbed TLS alapértelmezés szerint POSIX kompatibilis asztali operációs rendszerekre (például Linuxra) van optimalizálva, az `mbedtls_config.h` konfigurációs állományban jelentős módosításokat kellett végrehajtani a bare-metal (operációs rendszer és fájlrendszer nélküli) környezethez való illesztés érdekében.

A sikeres integráció és a stabilitás biztosítása érdekében az alábbi főbb beállításokat alkalmaztuk:

1. POSIX függőségek és operációs rendszer szintű funkciók eltávolítása

A fordítási hibák (hiányzó rendszerhívások, mint például az `open()`, `read()`, vagy socket API-k) elkerülése végett a következő platformfüggő funkciókat makrók szintjén letiltottuk:

- **Fájlrendszer I/O (`MBEDTLS_FS_IO`)**: Kikapcsolása elengedhetetlen, mivel a Secure Enclave nem rendelkezik hagyományos fájlrendszerrel a tanúsítványok és kulcsok beolvasásához.

– **Hálózati réteg (`MBEDTLS_NET_C`)**: A hálózati kommunikációt az architektúrában a Non-Secure World-höz csatolt ESP32 proxy modul végzi, így a kriptográfiai motornak nincs szüksége TCP/IP stackre.

– **Időkezelés (`MBEDTLS_HAVE_TIME`)**: Bár az X.509 tanúsítványok lejáratának ellenőrzéséhez szükséges lenne, a nyers ECDSA aláírási folyamat nem igényli a szabványos POSIX idő (epoch time) ismeretét.

– **Szálkezelés (`MBEDTLS_THREADING_C`)**: A mikrokontroller Secure World része szekvenciális végrehajtást alkalmaz, így a POSIX szálkezelő (pthread) és a beépített mutexek letiltásra kerültek.

2. Hardveres entrópia integrációja

Az Mbed TLS belső véletlenszám generátorának (Deterministic Random Bit Generator – CTR_DRBG) biztonságos seed-eléséhez elengedhetetlen a megfelelő minőségű entrópia. Mivel a `/dev/urandom` a beágyazott rendszeren nem áll rendelkezésre, az alapértelmezett operációs rendszeri entrópiát letiltottuk (`MBEDTLS_NO_PLATFORM_ENTROPY`). Helyette a hardveres alternatívát engedélyeztük (`MBEDTLS_ENTROPY_HARDWARE_ALT`), amely lehetővé tette az STM32U5 beépített, fizikai zajon alapuló hardveres véletlenszám-generátorának (TRNG) közvetlen használatát a kriptográfiai könyvtárban.

3. Platform absztrakciós réteg (`MBEDTLS_PLATFORM_C`)

Ezen makró engedélyezése biztosította azt a transzlációs réteget, amelyen keresztül a könyvtár memóriafoglalási (malloc/calloc) igényei biztonságosan átirányíthatóak lettek a beágyazott környezet saját heap kezelőjébe, megakadályozva a hiányzó szabványos C könyvtári (libc) hívásokból eredő linkelési hibákat.

3.3.2. A hitelesítési motor architektúrája

A Secure World fő feladata a hitelesítés megvalósítása. Mivel a rendszer támogatja mind a szimmetrikus, mind az aszimmetrikus kriptográfiát, a hitelesítési folyamat a kért módszertől függően a következő lépésekből áll:

Az algoritmus kritikus biztonsági tulajdonsága, hogy a firmware_start és firmware_size szimbólumokat a Secure World definiálja, így azok a Non-Secure kód által nem módosíthatók. Továbbá mind a szimmetrikus kulcs (K_{device}), mind az aszimmetrikus privát kulcs (d) szi-

1. Algorithm Secure World hitelesítési algoritmus (HMAC / ECDSA)

Require: nonce: verifier által küldött egyedi érték, $|\text{nonce}| = 4$ bájt
Require: timestamp: Non-Secure világ által megadott Unix időbélyeg
Require: auth_type: a kért hitelesítési mód (`HMAC` vagy `ECDSA`)
Ensure: signature: 32 bájtos HMAC hash VAGY 64 bájtos ECDSA (r, s) értékpár

```
firmware_start ← NS_FW_START_ADDR
firmware_size ← NS_FW_SIZE
H_f ← SHA256(firmware_start || firmware_size)
M ← H_f || timestamp || nonce
if auth_type = ECDSA then
    H_M ← SHA256(M)                          ▷ Az egyesített üzenet hash-e az aláíráshoz
    signature ← ECDSA_Sign(d, H_M)           ▷ d a védett privát kulcs
else
    signature ← HMAC-SHA256(K_device, M)     ▷ Szimmetrikus kulcs alapú hitelesítés
end if
return signature
```

gorúan a Secure World memóriájában van izolálva, ezáltal nem olvashatók ki egy esetlegesen kompromittált Non-Secure környezetből.

3.3.3. A hitelesítési jelentés és az aláírás generálása

A firmware mérésének és a hitelesítési válasz összeállításának folyamata a Secure World-ben történik. A Non-Secure kód kizárólag egy ellenőrzött kapun (Non-Secure Callable régió) keresztül hívhat Secure függvényeket [2.3.2]. A biztonságos átmenet biztosítása érdekében a megvalósított API függvények (amelyeket a `__attribute__((cmse_nonsecure_entry))` jelölővel láttunk el) szigorú memóriavédelmi ellenőrzéseket hajtanak végre a paraméterként kapott mutatókon.

A rendszer jelenlegi architektúrája kétféle validációs módszert támogat: a szimmetrikus HMAC-SHA256 [2.2.2], valamint az aszimmetrikus ECDSA eljárást [2.2.3].

A mutatók ellenőrzésére a TrustZone-M API `cmse_check_pointed_object` és `cmse_check_address_range` függvényeit használjuk, amelyek garantálják, hogy a Non-Secure világ ne kényszeríthesse a Secure World-öt védett memóriaterületek felülírására vagy kiolvasására [4.1.2].

A szimmetrikus (HMAC) hitelesítési jelentést generáló kód az alábbiak szerint épül fel:

```
__attribute__((cmse_nonsecure_entry))
int32_t SECURE_GenerateHMACReport(const attestation_report_t *report,
                                  uint8_t *hmac_out)
{
```

```
    /* Mutatok biztonsagi ellenorzese */
    const attestation_report_t *report_ns =
        (const attestation_report_t *)cmse_check_pointed_object(report,
        ↪  CMSE_NONSECURE);
    uint8_t *hmac_ns =
        (uint8_t *)cmse_check_address_range(hmac_out, ATTESTATION_HMAC_SIZE,
        ↪  CMSE_NONSECURE);

    if (report_ns == NULL || hmac_ns == NULL) return -1;

    /* Adatok atmasolasa Secure memoriaba a TOCTOU tamadasok ellen */
    attestation_report_t secure_copy;
    memcpy(&secure_copy, report_ns, sizeof(attestation_report_t));

    if (secure_copy.status != 0U) {
        memset(hmac_ns, 0, ATTESTATION_HMAC_SIZE);
        return -2;
    }

    /* Uzenet szerializalasa (44 bajt) */
    uint8_t message[44] __attribute__((aligned(4)));
    memset(message, 0, sizeof(message));
    memcpy(message, secure_copy.hash, 32);
    memcpy(message + 32, &secure_copy.nonce, 4);
    memcpy(message + 36, &secure_copy.timestamp, 4);
    message[40] = secure_copy.status;

    /* HMAC szamitas */
    int32_t result = compute_hmac_sha256(message, sizeof(message), hmac_ns);

    /* Erzekeny adatok torlese a memoriabol visszateres elott */
    memset(message, 0, sizeof(message));
    return (result != 0) ? result : 0;
}
```

Alternatívaként a rendszer támogatja az aszimmetrikus kulcsokon épülő ECDSA aláírást is, amely egy robusztusabb modellt biztosít, mivel a verifikátornak nem kell ismernie az eszköz titkos kulcsát a hitelesítéshez. A publikus kulcsot egy dedikált endpointon keresztül kérhetünk le, amelyet a szerver automatikusan lement. Ennek megvalósítása az `mbedtls` kriptográfiai könyvtár felhasználásával történik:

```
__attribute__((cmse_nonsecure_entry))
int32_t SECURE_GenerateECDSAReport(const attestation_report_ecdsa_t *report,
                                   uint8_t *sig_out, size_t max_sig_len,
                                   size_t *actual_sig_len)
{
    /* ... mutatok ellenorzese (cmse_check_...) es adatok atmasolasa ... */

    /* Szerializacio egy 44 bajtos tombbe (Nonce || Timestamp || Status || Hash) */
    uint8_t message[44];
    memcpy(message, &secure_copy.nonce, 4);
    memcpy(message + 4, &secure_copy.timestamp, 4);
```

```
    message[8] = secure_copy.status;
    memcpy(message + 12, secure_copy.hash, 32);

    /* Az ECDSA alairashoz szukseges a payload lenyomatanak (hash)kepzese */
    uint8_t message_hash[32];
    if (compute_hash_hw(message, sizeof(message), message_hash) != 0) return -3;

    /* mbedTLS kontextusok es RNG inicializalasa */
    mbedtls_ecdsa_context ecdsa;
    mbedtls_ctr_drbg_context ctr_drbg;
    // ... kontextusok beallitasa es privat kulcs betoltese
    ↪  (DEVICE_ECDSA_PRIVATE_KEY) ...

    /* ECDSA alairas generalasa */
    uint8_t temp_sig[100];
    size_t sig_len = 0;
    int32_t final_ret = mbedtls_ecdsa_write_signature(&ecdsa, MBEDTLS_MD_SHA256,
                                            message_hash, sizeof(message_hash),
                                            temp_sig, sizeof(temp_sig), &sig_len,
                                            mbedtls_ctr_drbg_random, &ctr_drbg);

    if (final_ret == 0 && sig_len <= max_sig_len) {
        memcpy(sig_ns, temp_sig, sig_len);
        *len_ns = sig_len;
    }

    /* Biztonsagi torles (Sanitization) */
    mbedtls_ecdsa_free(&ecdsa);
    memset(message_hash, 0, sizeof(message_hash));
    memset(temp_sig, 0, sizeof(temp_sig));

    return final_ret;
}
```

3.4. Non-Secure World-beli kommunikáció

A hálózati kommunikáció egy UART-n csatlakoztatott külső Wi-Fi modul segítségével valósul meg (ESP32 jelen esetben). A Non-Secure World kizárólagos feladata a hálózati stack kezelése, a bejövő JSON kérések (`nonce` kivonata) feldolgozása, naplózás, és az eredmény továbbítása a szerver felé.

3.5. Hálózati kommunikáció és ESP32 proxy

A rendszer hálózati elérését egy ESP32 modul biztosítja, amely `WiFi.h` és `WebServer.h` könyvtárak segítségével valósítja meg a HTTP végpontokat. Az ESP32 proxyként szolgál, amely a REST API-n érkező kéréseket UART interfészen keresztül továbbítja az STM32U5

mikrokontroller felé.

3.5.1. DoS védelem és Rate Limiting

A hálózati réteg rendelkezik egy alapvető *Rate Limiting* mechanizmussal a DoS támadások elkerülése érdekében. A `handle_attest` függvény vizsgálja az utolsó hitelesítési kérés időpontját, és 5 másodpercen belül érkező ismételt kéréseket HTTP 429 (*Too Many Requests*) hibakóddal utasítja el.

```
void handle_attest() {
  uint32_t now = millis();
  // DDoS/Rate limit védelem
  if ((now - last_attest_ms) < RATE_LIMIT_MS) {
    server.send(429, "application/json", "{\"error\":\"too_many_requests\"}");
    return;
  }
  last_attest_ms = now;
  // ... UART kommunikáció és válasz továbbítása
}
```

3.1. kódrészlet. Rate limiting megvalósítása az ESP32 proxyban

A kommunikáció során a rendszer fix 128 bájtos (`attest_response_t`) válaszcsomagokat vár a mikrokontrollertől, amelyeket az ESP32 a `read_exact` segédfüggvény segítségével, 8 másodperces időtúllépéssel (`UART_TIMEOUT_MS`) olvas ki. A beérkező bináris adatokat a szerver a JSON-formátumú HTTP válaszba történő továbbítás előtt hexadecimálissá alakítja át.

3.6. Python szerver oldal

A szerveroldali architektúra egy REST API-ból, egy hitelesítést ellenőrző modulból és egy adatbázis rétegből áll. Az API fogadja az eszköztől a kérést, kiszűri a túl régi időbélyegeket (replay támadás elleni védelem első rétege), majd meghívja a kriptográfiai a verifikálót.

3.6.1. Hitelesítő modul

A mikrokontroller által generált hitelesítési válaszok feldolgozásáért és kriptográfiai ellenőrzéséért a Python (Flask) alapú backend felelős. Amikor a hitelesítő megkapja az adatokat (nonce, időbélyeg, státusz, firmware hash és maga az aláírás), első lépésként a kötelező mezők meglétét és az adatok integritását (pl. elvárt nonce egyezése, státuszbájt érvényessége) vizsgálja.

Ezt követően a rendszer a kért hitelesítési módtól függően (`hmac` vagy `ecdsa`) újraépíti a 44 bájtos nyers bináris üzenetet. Kritikus fontosságú, hogy a Python kód pontosan ugyanabban a sorrendben (Little-Endian) fűzze össze az adatokat, mint ahogyan azt a TrustZone-M Secure World C kódja tette.

```python
def verify_attestation(report: dict, expected_nonce: int, attest_type: str, device: dict) -> dict:
    required = ["status", "nonce", "timestamp", "hash", "auth_data", "auth_len"]

    for field in required:
        if field not in report:
            return {"valid": False, "reason": f"missing field: {field}", "nonce": expected_nonce}

    try:
        status = int(report["status"])
        nonce = int(report["nonce"])
        timestamp = int(report["timestamp"])
        fw_hash = bytes.fromhex(report["hash"])
        auth_data = bytes.fromhex(report["auth_data"])
    except Exception:
        return {"valid": False, "reason": "invalid field encoding", "nonce": expected_nonce}

    if nonce != expected_nonce:
        return {"valid": False, "reason": "nonce mismatch", "nonce": nonce, "status": status, "timestamp": timestamp}

    verification_result = {
        "valid": False,
        "reason": "unknown algorithm",
        "status": status,
        "nonce": nonce,
        "timestamp": timestamp,
        "hash": fw_hash.hex(),
        "auth_data": auth_data.hex(),
    }

    # Firmware hash ellenőrzése
    expected_hash = device.get("expected_hash")
    if expected_hash and not hmac.compare_digest(fw_hash.hex(), expected_hash.lower()):
        verification_result["reason"] = "firmware hash mismatch"
        return verification_result

    # HMAC validációs ág
    if attest_type == "hmac":
        # 44 bájtos üzenet újraépítése: Hash || Nonce || Timestamp || Status || Padding
        msg = fw_hash + struct.pack("<I", nonce) + struct.pack("<I", timestamp) + bytes([status, 0, 0, 0])

        key = bytes.fromhex(device["hmac_key_hex"])
        expected_hmac = hmac.new(key, msg, hashlib.sha256).digest()
```

```
        if hmac.compare_digest(auth_data, expected_hmac):
            verification_result["valid"] = True
            verification_result["reason"] = "ok"
        else:
            verification_result["reason"] = "hmac mismatch"

    # ECDSA validációs ág
    elif attest_type == "ecdsa":
        pub_hex = device.get("ecdsa_pubkey_hex")
        if not pub_hex:
            verification_result["reason"] = "no public key registered"
            return verification_result

        pub_bytes = bytes.fromhex(pub_hex)

        # 44 bájtos üzenet újraépítése: Nonce || Timestamp || Status || Padding ||
        ↪ Hash
        msg = struct.pack("<I", nonce) + struct.pack("<I", timestamp) +
        ↪ bytes([status, 0, 0, 0]) + fw_hash

        try:
            # Publikus kulcs betöltése (SECP256R1)
            vk = VerifyingKey.from_string(pub_bytes[1:], curve=NIST256p)
            msg_hash = hashlib.sha256(msg).digest()

            # ASN.1 DER formátumú aláírás verifikálása
            if vk.verify_digest(auth_data, msg_hash, sigdecode=util.sigdecode_der):
                verification_result["valid"] = True
                verification_result["reason"] = "ok"
            else:
                verification_result["reason"] = "ecdsa signature math mismatch"
        except Exception as e:
            verification_result["reason"] = f"ecdsa verification failed: {str(e)}"

    return verification_result
```

3.6.2. SQLite adatbázis séma

A hitelesítési eredmények tartós tárolásáért és az eszközök menedzseléséért egy **SQLite** relációs adatbázis-kezelő (RDBMS) felel. A jelenlegi kliens-szerver architektúra megköveteli mind a szimmetrikus (HMAC), mind az aszimmetrikus (ECDSA) validációs folyamatok strukturált támogatását. Emiatt a mérési naplókat két elkülönített táblára bontottuk.

A megvalósított adatbázis-séma teljes SQL definíciója a következő:

```
-- Eszkozok regisztraciojat es kriptografiai kulcsait tarolo tabla
CREATE TABLE IF NOT EXISTS devices (
id INTEGER PRIMARY KEY AUTOINCREMENT,
device_id TEXT NOT NULL UNIQUE,
name TEXT NOT NULL,
base_url TEXT NOT NULL,
```

```
hmac_key_hex TEXT NOT NULL,
ecdsa_pubkey_hex TEXT,
expected_hash TEXT,
created_at INTEGER NOT NULL DEFAULT (unixepoch())
);

-- HMAC-SHA256 alapu hitelesitesi meresek naploja
CREATE TABLE IF NOT EXISTS hmac_attestations (
id INTEGER PRIMARY KEY AUTOINCREMENT,
device_id TEXT NOT NULL,
nonce INTEGER NOT NULL,
status INTEGER,
timestamp INTEGER,
firmware_hash TEXT,
received_hmac TEXT,
expected_hmac TEXT,
valid INTEGER NOT NULL,
reason TEXT NOT NULL,
duration_ms REAL,
created_at INTEGER NOT NULL DEFAULT (unixepoch()),
FOREIGN KEY (device_id) REFERENCES devices(device_id)
);

-- ECDSA alapu hitelesitesi meresek naploja
CREATE TABLE IF NOT EXISTS ecdsa_attestations (
id INTEGER PRIMARY KEY AUTOINCREMENT,
device_id TEXT NOT NULL,
nonce INTEGER NOT NULL,
status INTEGER,
timestamp INTEGER,
firmware_hash TEXT,
signature TEXT,
valid INTEGER NOT NULL,
reason TEXT NOT NULL,
duration_ms REAL,
created_at INTEGER NOT NULL DEFAULT (unixepoch()),
FOREIGN KEY (device_id) REFERENCES devices(device_id)
);
```

3.6.3. Felhasználói felület

Áttekintés

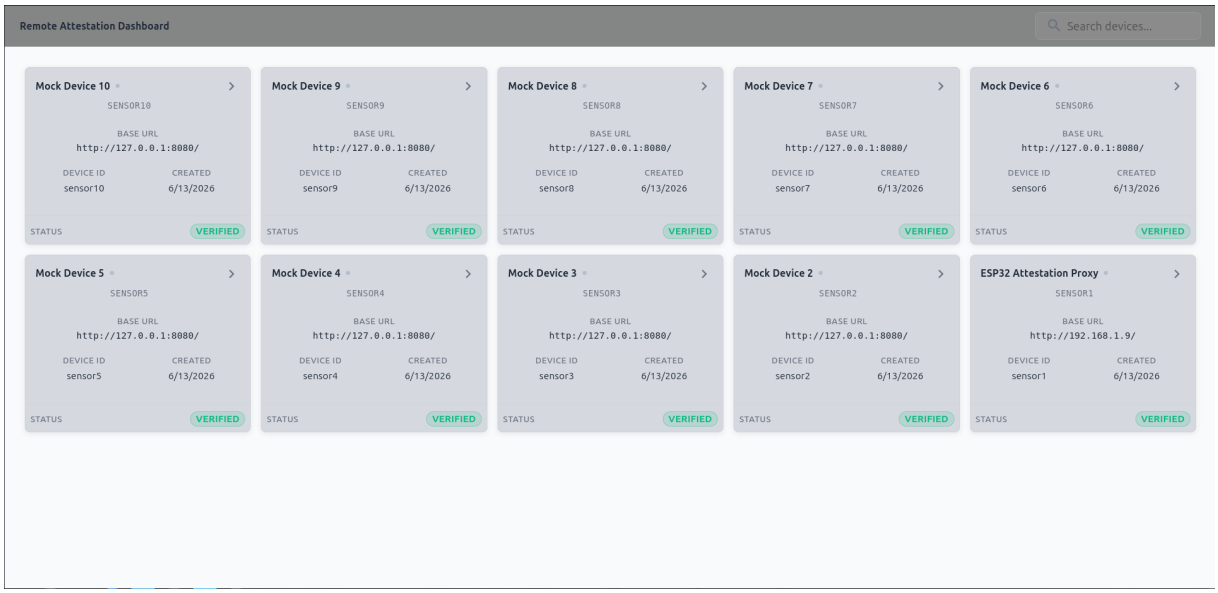
A hitelesítési rendszer menedzselésére és az IoT végpontok állapotának nyomon követésére egy modern, egyoldalas webalkalmazás (Single Page Application – SPA) készült. A kliensoldali felület React és TypeScript alapokon nyugszik, a gyors fordítást és modulkezelést pedig a Vite keretrendszer biztosítja. A felhasználói felület (UI) vizuális elemeinek és reszponzív rácsszerkezetének kialakításához a Material-UI (MUI) komponenskönyvtárat alkalmaztuk. A frontend a React Router segítségével valósítja meg az oldalak közötti kliensoldali navigációt, míg a Flask

backenddel történő aszinkron REST API kommunikációért (például a hitelesítési kérések indításáért és a naplók lekéréséért) az Axios HTTP kliens felel.

Nézetek és funkciók

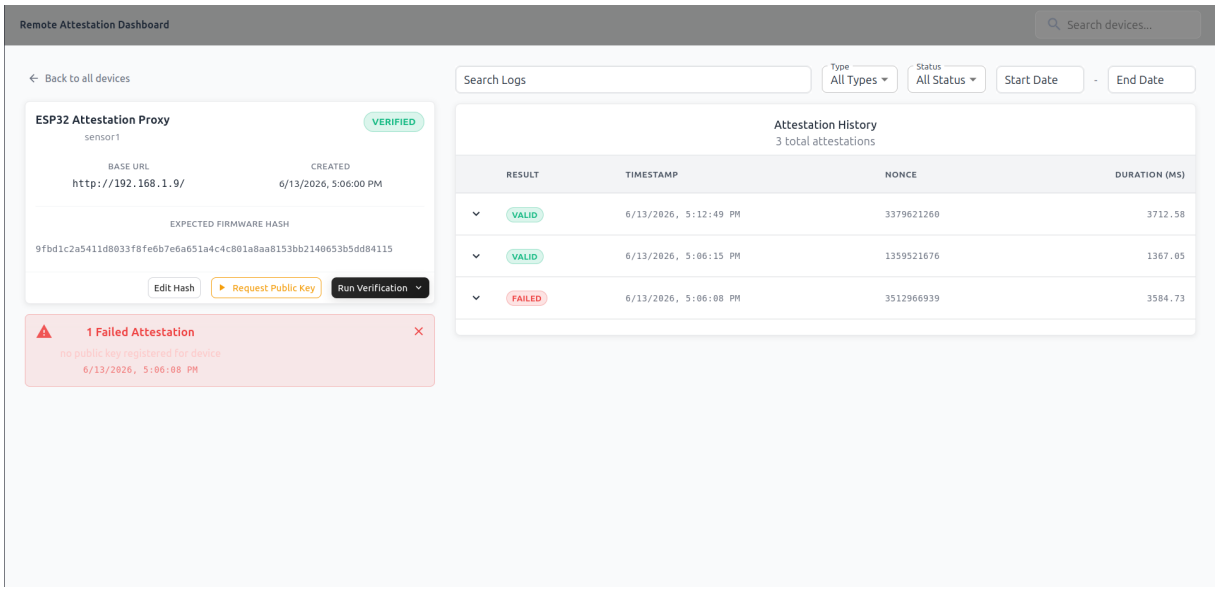
A webes dashboard négy fő nézetre tagolódik, amelyek végigkísérik a felhasználót az eszközök felügyeletétől a konkrét kriptográfiai műveletek indításáig és ellenőrzéséig.

Eszközök listája A webalkalmazás főoldala az összes mikrokontroller áttekintésére szolgál. A nézet egyenként az eszköz IP címét, ID-ját, létrehozási dátumát és a legutóbbi hitelesítési kísérletünk státuszát mutatja (megbízható vagy kompromittált).



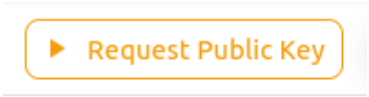
3.1. ábra. Az eszközök listájának megjelenítése az irányítópulton

Eszköz részletes oldala A listából egy konkrét elemet kiválasztva a felhasználó az eszköz részletes adataihoz lép. Itt jelenik meg az elvárt firmware hash, illetve az eddigi hitelesítések. Ezeket tudjuk szűrni is dátum, státusz és az algoritmus típusa szerint is. Ezen a felületen keresztül érhetők el az adott eszközre vonatkozó specifikus műveletek is. Az esetleges sikertelen hitelesítésről külön figyelmeztet az oldal.



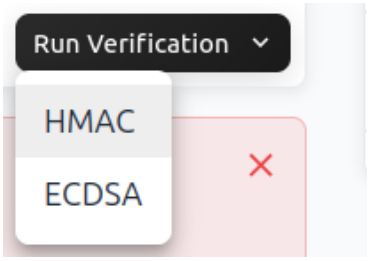
3.2. ábra. Az IoT eszköz részletes nézete

Publikus kulcs kérése Az ECDSA hitelesítés során használt publikus kulcsot lekérhetjük egy gomb lenyomásával. Ez automatikusan lementődik az adatbázisba.



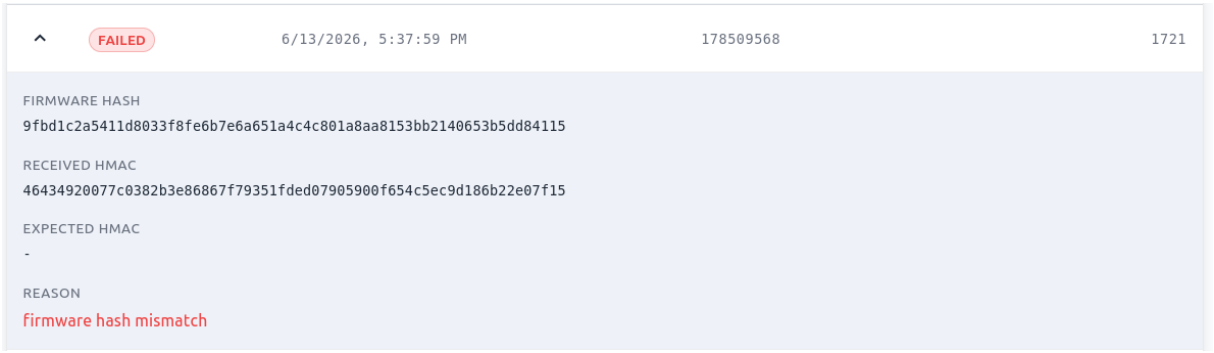
3.3. ábra. Publikus kulcs kérése

Hitelesítés végrehajtása A rendszer legfontosabb interakciója a hitelesítés indítása. A felhasználó ezen keresztül választhatja ki az alkalmazni kívánt kriptográfiai eljárást (szimmetrikus HMAC-SHA256 vagy aszimmetrikus ECDSA). A művelet elindításakor a frontend egy aszinkron kérést küld a szervernek, és egy töltőképernyővel (loading state) jelzi a folyamatban lévő hitelesítést.

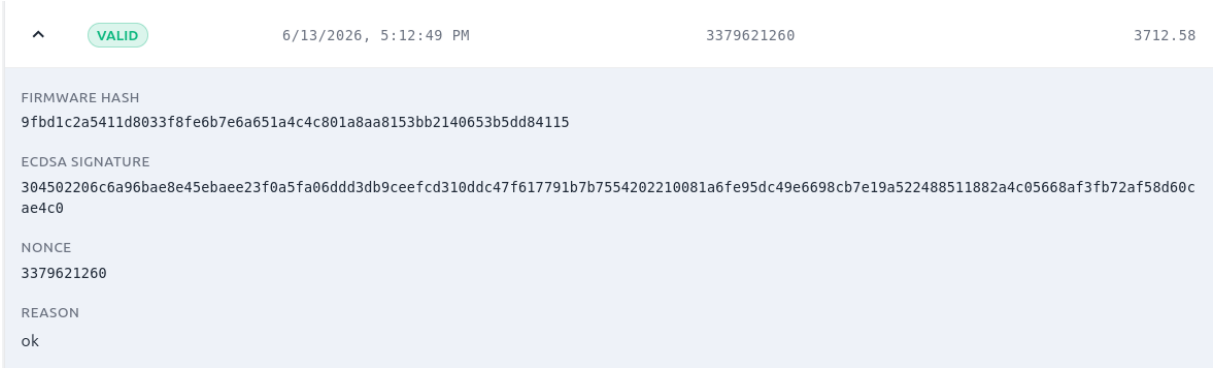


3.4. ábra. A hitelesítés indításához használt gomb

Hitelesítési előzmények A hálózat biztonsági eseményeinek visszakövethetőségét a naplózó felület biztosítja, amely az adatbázis tartalmát jeleníti meg. Ebben a nézetben időrendi sorrendben láthatók a korábbi hitelesítési kísérletek, melyek lenyitásakor külön megtekinthetők a használt algoritmusok, az időbélyegek, valamint az esetleges hibakódok (például időtúllépés, érvénytelen aláírás vagy kompromittálódott hash).



3.5. ábra. Sikertelen hitelesítés



3.6. ábra. Sikeres hitelesítés

4. fejezet

Eredmények értékelése és konklúzió

4.1. Implementációs tapasztalatok

A hitelesítési rendszer sikeres megvalósítása az STM32U575 TrustZone-M platformon bebizonyította, hogy a hardver szintű biztonsági mechanizmusok gyakorlatban alkalmazhatók az IoT területén. A Secure/Non-Secure szeparáció valóban megvalósítható és hatékony, ugyanakkor a fejlesztési folyamat során számos nem triviális kihívással kellett szembenézni.

4.1.1. TrustZone-M projekt konfiguráció

A TrustZone-M fejlesztés egyik legnagyobb kihívása a kétprojektes munkakörnyezet helyes beállítása volt. Az STM32CubeIDE külön projektet kezel a Secure és a Non-Secure firmwarenek, amelyeket egymás után kell lefordítani és flashelni – a Secure projektet mindig először, mivel az tartalmazza az SAU és GTZC konfigurációt, amelytől a Non-Secure oldal indulása függ.

Az Secure World beli konfiguráció különösen érzékeny: a hashelt memória címe és mérete pontosan egyeznie kell a Non-Secure projekt flash kiosztásával. Eltérés esetén a firwmare hash értéke nem fog megfelelni az elvártaknak, vagy lehetőséget ad egy olyan memóriterület felülírására ami nincs felügyelve.

A firmware mérési tartományának meghatározása kizárólag a Secure World-ben történik. A korai prototípusokkal szemben a Secure komponens nem támaszkodik a Non-Secure linker script által exportált szimbólumokra, jelen esetben `NS_FIRMWARE_START` és `NS_FIRMWARE_SIZE`, mivel ezek az attesztáció szempontjából nem tekinthetők megbízható forrásnak. Ehelyett a Non-Secure flash tartomány kezdőcíme és mérete a Secure projektben kerül rögzítésre. A hash számítás előtt a Secure World a TrustZone hardveres adottságainak segítségével ellenőrzi, hogy a megadott tartomány valóban Non-Secure memóriára mutat. Ez biztosítja, hogy a mérés mindig ugyanarra a firmware régióra történjen, függetlenül a Non-Secure alkal-

mazás konfigurációjától.

4.1.2. NSC veneer pointer biztonság

A Non-Secure Callable kapu implementálásakor a `cmse_check_address_range()` ellenőrzés bevezetése nem nyilvánvaló szükségszerűség első ránézésre, de kritikus fontosságú. Egy korai tesztelési fázisban, az ellenőrzés nélküli verzióban a Non-Secure kód tévesen Secure flash-re mutató pointert adott át. Ez a hiba a *confused deputy* sérülékenység egy konkrét példánya volt, és csak azért nem okozott problémát, mert a tesztelés során a Secure flash releváns területe nullával volt feltöltve. Az ellenőrzés bevezetése után ez a hibaosztály teljesen megszűnt.

4.1.3. Hardver akcelerátor megszakíthatósága

Felmerülő probléma lehet a hash számítás megszakítása a végrehajtás során. Az ARM TrustZone-M architektúra nem teszi lehetővé a Secure és Non-Secure állapot egyidejű futtatását ugyanazon processzor magon, hanem kontextusváltáson alapul. Emellett a vizsgált mikrokontroller egy egymagos rendszer, így valódi párhuzamos végrehajtás nem lehetséges.

Az implementáció a HASH hardveres gyorsítót egyetlen Secure végrehajtási kontextusból használja. Mivel a prototípus nem alkalmaz valós idejű operációs rendszert (RTOS - Real-Time Operating System), és nincs több, párhuzamosan futó Secure szolgáltatás, a HASH periféria egyidejű elérése nem merül fel.

A hash művelet körüli megszakítás maszkolást (*interrupt masking*) kizárólag reentrancia és megszakítás alapú inkonzisztens állapotok megelőzésére értékeltük, jövőbeli kiterjesztések esetére, ahol összetettebb Secure végrehajtási környezet vagy RTOS kerül bevezetésre.

4.2. Teljesítményi adatok összegzése

SHA256 számítási sebesség

Az STM32U5 beépített hardver kriptográfiai akcelerátor több mint kétszeres gyorsulást biztosít.

Implementáció	Firmware méret	Idő
Szoftveres SHA256	1 MB	2 s
Hardver-gyorsított SHA256	1 MB	0.8 s

4.1. táblázat. SHA256 teljesítményadatok

Hitelesítési válaszidő (HMAC alapú folyamat)

A teljes hitelesítési folyamat end-to-end ideje a firmware hash számítást, a hitelesítési kulcs-alapú aláírást (HMAC), valamint a hálózati továbbítást is magában foglalja:

$$T_{\text{total,HMAC}} = T_{\text{hash}} + T_{\text{hmac}} + T_{\text{network}} + T_{\text{overhead}} \tag{4.1}$$

A mérések alapján:

- $T_{\text{hash}} = 800$ ms (firmware hash számítás, hardveres SHA256 gyorsítással)
- $T_{\text{hmac}} = 50$–100 ms (HMAC-SHA256 számítás és előkészítés)
- $T_{\text{network}} = 300$–800 ms (Wi-Fi késleltetés és átvitel)
- $T_{\text{overhead}} = 200$–$300$ ms (adatcsomagolás, periféria-kezelés, protokoll overhead)

Összesített válaszidő: 1.6–1.8 s

Hitelesítési válaszidő (ECDSA alapú folyamat)

ECDSA-alapú hitelesítés esetén a kriptográfiai művelet számításigénye jelentősen magasabb, mivel aszimmetrikus aláírást használ:

$$T_{\text{total,ECDSA}} = T_{\text{hash}} + T_{\text{ecdsa}} + T_{\text{network}} + T_{\text{overhead}} \tag{4.2}$$

A mérések alapján:

- $T_{\text{hash}} = 800$ ms (firmware hash számítás, hardveres SHA256 gyorsítással)
- $T_{\text{ecdsa}} = 1.9$–2.2 s (ECDSA aláírási művelet)
- $T_{\text{network}} = 300$–800 ms (Wi-Fi késleltetés és átvitel)
- $T_{\text{overhead}} = 200$–$300$ ms (protokoll és rendszer overhead)

Összesített válaszidő: 3.5–3.8 s

4.2.1. Biztonsági értékelés

Támadási felületek és mitigálásuk

1. *Meghatározás* (Támadási felület). Az a halmaz, amellyel egy támadó potenciálisan interakcióba léphet a rendszerrel annak biztonsági megsértésére.

1. **Firmware módosítás támadás**
   - *Veszély*: Támadó megváltoztatja a Non-Secure firmware-t
   - *Védelmi intézkedés*: A hitelesítés azonnal felderíti az eltérő hash-t
2. **Replay támadás**
   - *Veszély*: Régi, érvényes hitelesítési válaszokat játszik vissza
   - *Védelmi intézkedés*: Időbélyeg + nonce validálása szerveroldalon
3. **Titkos kulcs kompromittálása**
   - *Veszély*: Ha a Secure Flash-t valahogy kiolvassák
   - *Védelmi intézkedés*: TrustZone-M hardverszintű védelme
4. **Confused deputy támadás**
   - *Veszély*: Non-Secure kód Secure memóriacímet ad át az NSC kapunak
   - *Védelmi intézkedés*: `cmse_is_nonsecure_address()` ellenőrzés az NSC veneerben
5. **Denial-of-Service (DoS) támadás**
   - *Veszély*: A támadó hálózati kérésekkel bombázza az eszközt, lefoglalva a Secure World számítási kapacitását.
   - *Védelmi intézkedés*: Hálózati Rate Limiting bevezetése (bővebben lásd 3.5.1. fejezet).

1. *Hitelesítési biztonsági garancia.* Ha a hitelesítés helyesen verifikálódik, a firmware integritása garantált, feltéve hogy a Secure World memóriája és a titkos kulcs sértetlenül maradt.

Bizonyítás. A hitelesítés az alábbi biztonsági összetevőkön alapul: (1) firmware hash közvetlenül Secure World-ből, (2) titkos kulcs kizárólag Secure memóriában tárolva, (3) HMAC-SHA256 kriptográfiai biztonsága (preimage + ütközésrezisztencia). Ha az HMAC verifikálódik, az aláíráshoz szükséges összetevők (titkos kulcs, üzenet) azonosnak kell lenniük. Ezáltal a firmware hash is garantáltan azonos a szerveroldali verzióval. ■

Megjegyzés A rendszer alapvető feltételezése, hogy a TrustZone-M hardverszintű szeparációja valóban megbízható. Ez az ARM processzorok tervezett és iparban elfogadott tulajdonsága.

4.2.2. Mérési módszertan

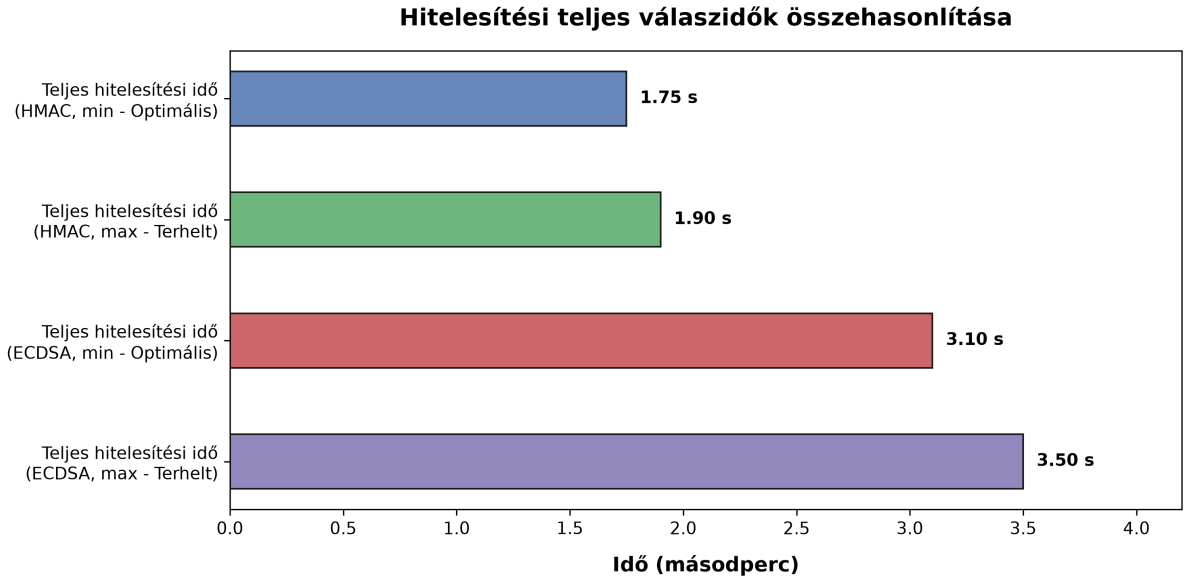
A teljesítménymérések a következő módszerekkel kerültek elvégzésre:

- **Firmware hash ideje**: A STM32U5 `DWT` (Data Watchpoint and Trace) ciklusszámláló segítségével történt a mérés, majd az eredmény CPU óraciklusból milliszekundumba lett konvertálva (160 MHz órajel).
- **HMAC-SHA256 ideje**: Szintén DWT alapú mérés a `hmac_sha256()` függvény körül.
- **Hálózati késleltetés**: A Python szerver oldalon `time.perf_counter()` alapú mérés.

Minden mérés 100-szor lett elvégezve, és az átlag került rögzítésre, kiszűrve az outliereket (±3 szórásnyi eltérés).

Metrika	Érték	Megjegyzés
Firmware hash számítás (HW)	800 ms	1 MB firmware, 160 MHz, fix idő
Firmware hash számítás (SW)	2,00 s	Referencia, összehasonlításképp
Hardveres gyorsulás	2,5×	HW vs SW arány
HMAC-SHA256 generálás	550–600 ms	Hardveres implementáció
ECDSA P-256 aláírás	1,90–2,20 s	Aszimmetrikus kriptográfiai művelet
NSC átmenet overhead	$<$1 ms	Secure/Non-Secure váltás
Teljes eszközoldali feldolgozás (HMAC)	1,35–1,40 s	Hash (HW) + HMAC (SW) + overhead
Teljes eszközoldali feldolgozás (ECDSA)	2,70–3,00 s	Hash (HW) + ECDSA (SW) + overhead
Kommunikációs overhead (min)	395 ms	SPI busz + Wi-Fi stack (optimális eset)
Kommunikációs overhead (max)	495 ms	SPI busz + hálózati késleltetés (terhelt eset)
Szerveroldali feldolgozás	5 ms	Verifikáció + DB írás
Teljes hitelesítési idő (HMAC, min)	**1,75 s**	Optimális körülmények
Teljes hitelesítési idő (HMAC, max)	**1,90 s**	Maximális időigény terhelés alatt
Teljes hitelesítési idő (ECDSA, min)	**3,10 s**	Optimális körülmények
Teljes hitelesítési idő (ECDSA, max)	**3,50 s**	Maximális időigény terhelés alatt

4.2. táblázat. Rendszer teljesítményének hiteles, mért adatai (1 MB firmware)



4.1. ábra. Algoritmusok összehasonlítása

Energiafogyasztás és üzemidő becslés

A rendszer energiaigényének és várható üzemidejének meghatározásához különböző hitelesítési gyakoriságokat (5, 10, 30 és 60 perc) vizsgáltunk. A számításokat tisztán az STM32U5 mikrokontroller hivatalos gyártói adatlapja [16], valamint az előzőekben bemutatott, valós mé-

réseken alapuló hitelesítési futásidők alapján végeztük el. Ebben a becslésben a hálózati kommunikációs modul (pl. Wi-Fi) energiaigényét nem vettük figyelembe, emellett valós környezetben más tényezők is befolyásolhatják az energiafogyasztást.

Az elemzés során a következő paraméterekkel és feltételezésekkel számoltunk:

– **Működési feszültség és SMPS tápellátás:** A mikrokontroller 3.3 V-os tápfeszültségen üzemel. A referenciaként választott 2500 mAh kapacitású Li-ion akkumulátor feszültsége azonban 3.7 V. Mivel a rendszer az STM32U5 beépített, hatékony tápegységét (SMPS) használja, a fogyasztást teljesítmény alapon (mWh) vizsgáltuk. Az akkumulátorból felvett áramot $I_{batt} = \frac{U_{mcu} \cdot I_{mcu}}{U_{batt} \cdot \eta}$ képlettel határoztuk meg, $\eta = 0.9$ (90%-os) átlagos hatékonyságot feltételezve.

– **Aktív mód (160 MHz):** Az adatlap alapján az MCU fogyasztása 19.5 μA/MHz, így a futási fogyasztás 3.3 V-on ≈ 3.12 mA. (Az akkumulátor oldalán ez ≈ 3.09 mA).

– **Idle mód (Stop 2, teljes SRAM megtartásával):** A hitelesítések közötti inaktív állapotban a fogyasztás 3.3 V-on mindössze ≈ 9 μA (0.009 mA).

– **Hitelesítési idő:** A mérések alapján a feldolgozás HMAC esetén $\approx$ 1.7 másodpercet, ECDSA esetén pedig ≈ 3.65 másodpercet vesz igénybe.

Gyakoriság	Kriptográfia	Átlagos fogyasztás	Elméleti üzemidő (2500 mAh)
5 perc	HMAC-SHA256	0.0264 mA	≈ 10.8 év
	ECDSA P-256	0.0464 mA	≈ 6.1 év
10 perc	HMAC-SHA256	0.0176 mA	≈ 16.2 év
	ECDSA P-256	0.0277 mA	≈ 10.3 év
30 perc	HMAC-SHA256	0.0118 mA	≈ 24.1 év
	ECDSA P-256	0.0152 mA	≈ 18.8 év
60 perc	HMAC-SHA256	0.0104 mA	≈ 27.5 év
	ECDSA P-256	0.0120 mA	≈ 23.7 év

4.3. táblázat. Az akkumulátort terhelő átlagos fogyasztás és elméleti üzemidő SMPS használata esetén (csak MCU)

Mint az a 4.3. táblázatból egyértelműen látszik, az aszimmetrikus ECDSA P-256 algoritmus alkalmazása nagyobb energiafogyasztással jár, mint a szimmetrikus HMAC-SHA256 eljárás. Ez a többlet közvetlenül a hosszabb eszközoldali feldolgozási időből adódik, ami sűrűbb (például 5 perces) hitelesítési ciklusok esetén csaknem a felére csökkenti az elméleti akkumulátor üzemidejét.

4.3. Biztonsági konklúziók

4.3.1. Elért biztonsági garanciák

A dolgozat során felépített hitelesítési rendszer három fő biztonsági garanciát nyújt:

1. **Firmware integritás**: A SHA256 hash közvetlenül a Secure World-ből számítódik, a Non-Secure kód nem képes befolyásolni a mérés eredményét.
2. **Hitelesítés**: Az HMAC-SHA256 és ECDSA-P256 aláírás garantálja az üzenet és a küldő hitelességét.
3. **Replay támadás megelőzés**: Az időbélyeg és a nonce kombinációja biztosítja, hogy egy elfogott, korábban érvényes hitelesítési válasz ne legyen újra felhasználható.
4. **DoS támadás elleni védelem**: A túl gyakori kéréseket visszautasítjuk.

4.3.2. Ismert korlátok

Titkos kulcs tárolása

A kulcsok tárolásának biztonsága mind a kliens, mind a szerver oldalon kritikus pontja a hitelesítési rendszernek.

A beágyazott eszköz tekintetében a jelen implementációban a titkos kulcs a Secure Flash-be *hardcode*-olt konstansként van beágyazva. Egy éles (production) rendszerben a hardveres biztonság növelése érdekében az egyszer írható OTP (One-Time Programmable) memória használata, valamint a Read-Out Protection (RDP) aktiválása szükséges a fizikai (például JTAG interfészen keresztüli) kiolvasás megelőzésére.

A szerveroldali tárolás esetében a jelenlegi prototípus a szimmetrikus HMAC hitelesítéshez használt kulcsot egyszerű szöveges (plaintext) formátumban tárolja az adatbázisban. A jövőbeli kiterjesztések során a szerveroldali kulcsokat adatbázisszintű titkosítással (encryption-at-rest), kulcskezelő szolgáltatásokkal (KMS), vagy egy dedikált hardveres biztonsági modulban (HSM / Secure Enclave) szükséges elhelyezni, megelőzve a szerveroldali kulcs kiszivárgását.

Hálózati kommunikáció titkosítása

A jelenlegi implementáció HTTP-n (nem HTTPS-en) kommunikál. Bár a hitelesítési aláírás maga védett, TLS (mbedTLS) hozzáadása a hálózati rétegen éles rendszerben szükséges lenne.

A TLS hiányát a jelenlegi hardverarchitektúra indokolja: a kommunikációért jelenleg egy dedikált ESP32 mikrokontroller felel, amely proxyként szolgál az STM32U5 mikrokontroller számára. A jövőbeli fejlesztések során célunk az ESP32-es proxy kiváltása, és a hálózati verem közvetlenül a mikrokontrolleren történő implementálása, amely lehetővé tenné a már rendelkezésre álló Mbed TLS könyvtár használatát a biztonságos hálózati kommunikációhoz.

Control-Flow Attesztáció korlátai

A megvalósított rendszer kizárólag statikus firmware integritás ellenőrzést végez, és nem valósít meg Control-Flow Attestation (CFA) mechanizmust.

A TrustZone-M architektúrára épülő STM32U5 mikrokontroller esetén a Secure és Non-Secure világ nem képes párhuzamos végrehajtásra. A Cortex-M33 processzor egy adott időpillanatban kizárólag az egyik biztonsági tartomány kódját hajtja végre, a két világ közötti váltás pedig kontextusváltással történik. Emiatt a Secure World nem képes a Non-Secure alkalmazás vezérlésfolyamának (control flow) valós idejű megfigyelésére annak futásával egyidejűleg.

További korlátot jelent, hogy az STM32U5 egyetlen processzormaggal rendelkezik, így nem áll rendelkezésre külön végrehajtási egység az attesztációs logika és a megfigyelt alkalmazás párhuzamos futtatására. Ennek következtében a jelen rendszer firmware integritás alapú távoli hitelesítést valósít meg, azonban futásidejű vezérlési útvonal hitelesítésére nem alkalmas.

4.4. Kiterjesztési lehetőségek

4.4.1. Over-the-Air (OTA) firmware frissítés

A Secure World természetes helye a biztonságos OTA frissítési folyamat kezelésének, ahol az új szoftver kriptográfiai aláírásának verifikálását a biztonságos partíció végzi, még mielőtt az a flash memóriába kerülne.

4.4.2. Többeszközös hitelesítés és fleet kezelés

A szerver oldal képes egy teljes eszközhálózat kezelésére ütemezett hitelesítések (polling), trendanalízis és anomália detektálás segítségével.

4.4.3. Alternatív hálózati protokollok: MQTT és CoAP

A jelenlegi implementáció a széles körben elterjedt HTTP (Hypertext Transfer Protocol) protokollt használja a hitelesítési adatok továbbítására. Bár a HTTP előnye az egyszerű megvalósíthatóság és a meglévő webes infrastruktúrába való könnyű integrálhatóság, egy erőforráskorlátozott IoT eszköz számára jelentős terhelést jelent.

A mérések során tapasztalt hálózati késleltetés ($T_{network}$) nagy részét a HTTP protokoll szövegalapú, redundáns fejlécei és a TCP kapcsolat felépítésének költsége teszi ki. Éles környezetben, ahol az energiafogyasztás és a sávszélesség kritikus, az alábbi protokollok jelentenek hatékonyabb alternatívát:

1. **MQTT (Message Queuing Telemetry Transport)**: Az MQTT egy könnyűsúlyú, bináris alapú protokoll. A fix fejléc mindössze 2 bájt, ami drasztikusan csökkenti a hálózati forgalmat a HTTP több száz bájtos fejléceihez képest. A hitelesítési folyamat során a szerver egy adott *topic*-ra küldhetné a nonce-ot (number used only once), az eszköz pedig egy válasz-topic-on publikálná az aláírt mérést, ami hatékonyabbá tenné a többeszközös (fleet) kezelést.

2. **CoAP (Constrained Application Protocol)**: A CoAP kifejezetten a legkisebb mikrokontrollerekre lett tervezve. UDP-t használ TCP helyett, így eltünteti a kapcsolatfelépítés várakozási idejét. A CoAP natívan támogatja a DTLS (Datagram Transport Layer Security) titkosítást, ami biztosíthatná a Non-Secure csatorna védelmét is.

Ezen protokollok bevezetése nemcsak a válaszidőt rövidítené le, hanem az eszköz rádiómoduljának aktív idejét is csökkentené, ami közvetlenül növelné az akkumulátoros üzemidőt.

4.4.4. Hordozhatóság más mikrokontrollerekre

A bemutatott hitelesítési architektúra bár specifikusan az STM32U5 mikrokontrolleren került megvalósításra, koncepcionálisan könnyedén átültethető más, ARM TrustZone-M támogatással rendelkező mikrokontrollerekre. Például az STMicroelectronics egy másik, szintén Cortex-M33 magra épülő architektúrája, az STM32L5 sorozat, vagy akár más gyártók (például NXP LPC55S69) hasonló eszközei is azonnal alkalmazhatók.

A portolás során a magas szintű C kód nagy része változatlanul hagyható. Csupán az alábbi, hardverspecifikus rétegeket kell az adott chiphez igazítani:

– A memóriatérkép és a TrustZone határok (SAU/IDAU) konfigurációját.

- A hardveres kriptográfiai gyorsítók (pl. `HAL_HASH`) inicializálását.

A rendszer magja: a hitelesítési protokoll, a Secure/Non-Secure Callable (NSC) kapu logikája, a pointer validáció, valamint a teljes Python szerveroldal hardverfüggetlen, így minimális módosítással újrahasznosítható.

Összefoglaló

A szakdolgozatban egy valós, működő hitelesítési rendszer került megtervezésre és implementálásra, amely az alábbi tulajdonságokkal rendelkezik:

- **Hardver-alapú biztonság**: A TrustZone-M szeparáció garantálja, hogy a kompromittált Non-Secure kód nem képes meghamisítani a hitelesítési eredményt.
- **Mérhető teljesítmény**: A hardver SHA256 akcelerátor 800 ms-os hash számítást tesz lehetővé 1 MB firmware esetén.
- **Teljes rendszerintegráció**: A beágyazott C firmware, a Python Flask backend, az SQLite adatbázis és a React weboldal együttesen egy végponttól végpontig terjedő megoldást alkot.
- **Hordozható architektúra**: Bár a referencia implementáció STM32U5 mikrokontrollerre készült, a rendszerterv és a szoftveres alapok (TrustZone-M veneer-ek, kliens-szerver protokoll) könnyedén implementálhatók más Cortex-M33 alapú, TrustZone képes eszközökre (például az STM32L5 sorozatra).

Az ARM TrustZone-M technológia lehetővé tette olyan biztonsági mechanizmusok megvalósítását, amelyek korábban csak nagyobb, dedikált biztonsági hardverrel (TPM, HSM) rendelkező rendszerekben voltak elérhetők. Ez a dolgozat megmutatja, hogy a modern beágyazott platform — megfelelő tervezéssel — valódi, kriptográfiailag megalapozott hardveres biztonságot nyújthat az IoT alkalmazások számára.

Irodalomjegyzék

[1] Arm Limited. „Arm TrustZone-M Technology Overview”. *Arm Architecture Technical White Paper* (2023). URL: https://www.arm.com/technologies/trustzone-for-cortex-m.

[2] Arm Limited. *Trusted Firmware-M (TF-M)*. https://developer.arm.com/Tools%20and%20Software/Trusted%20Firmware-M. [Online; letöltve: 2026. június 12.] 2026.

[3] Arm Limited and Partners. *PSA Certified - Platform Security Architecture*. https://www.psacertified.org/. 2026.

[4] Karim El Defrawy és mások. „SMART: Secure and Minimal Architecture for (Establishing a Dynamic) Root of Trust”. *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society. 2012. URL: https://www.ndss-symposium.org/wp-content/uploads/2017/09/10_3.pdf.

[5] Karim Eldefrawy, Norrathep Rattanavipanon és Gene Tsudik. „Hydra: Hybrid Design for Remote Attestation (Using a Formally Verified Microkernel)”. *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security (Asiacrypt)*. ACM, 2020, 99–112. oldal. URL: https://dl.acm.org/doi/10.1145/3098243.3098261.

[6] Hugo Krawczyk, Mihir Bellare és Ran Canetti. *HMAC: Keyed-Hashing for Message Authentication*. RFC 2104. Internet Engineering Task Force (IETF), 1997. febr. URL: https://datatracker.ietf.org/doc/html/rfc2104.

[7] Laurence Lundblade és mások. *The Entity Attestation Token (EAT)*. RFC 9334. Internet Engineering Task Force (IETF), 2023. URL: https://datatracker.ietf.org/doc/rfc9334/.

[8] Roland Nagy és mások. „Rootkit Detection on Embedded IoT Devices”. *Acta Cybernetica* 25.2 (2021), 311–330. oldal. URL: https://static.crysys.hu/publications/files/setit/cpaper_bme_NagyBPB21eurocybersec.pdf.

[9] National Institute of Standards and Technology (NIST). *Digital Signature Standard (DSS)*. Federal Information Processing Standards Publication FIPS PUB 186-5. U.S. Department of Commerce, 2023. febr. URL: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf.

[10] National Institute of Standards and Technology (NIST). *FIPS PUB 198-1: The Keyed-Hash Message Authentication Code (HMAC)*. Federal Information Processing Standards Publication. U.S. Department of Commerce, 2008. URL: https://csrc.nist.gov/publications/detail/fips/198/1/final.

[11] National Institute of Standards and Technology (NIST). *Secure Hash Standard (SHS)*. Federal Information Processing Standards Publication FIPS PUB 180-4. U.S. Department of Commerce, 2015. URL: https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.180-4.pdf.

[12] Sandro Pinto és Nuno Santos. „Demystifying Arm TrustZone: A Comprehensive Survey". *ACM Computing Surveys (CSUR)* (2019). URL: https://dl.acm.org/doi/10.1145/3291047.

[13] Remote Attestation Procedures (RATS) Working Group. *Remote ATtestation ProcedureS (RATS) Architecture*. RFC 9334. IETF, 2023. URL: https://datatracker.ietf.org/wg/rats/documents/.

[14] Reiner Sailer és mások. „Design and Implementation of a TCG-based Integrity Measurement Architecture". *Proceedings of the 13th USENIX Security Symposium*. 13. kötet. San Diego, CA, USA, 2004, 16. oldal. URL: https://www.usenix.org/legacy/publications/library/proceedings/sec04/tech/full_papers/sailer/sailer.pdf.

[15] Carlton Shepherd és mások. „Establishing Mutually Trusted Channels for Remote Sensing Devices with Trusted Execution Environments". *Proceedings of the International Conference on Remote Sensing and IoT Security*. Springer, 2017, 1–10. oldal. URL: https://dl.acm.org/doi/10.1145/3098954.3098971.

[16] STMicroelectronics. *STM32U575xx Datasheet (DS13737 Rev 10)*. Production Data. STMicroelectronics. 2024. URL: https://www.st.com/resource/en/datasheet/stm32u575ag.pdf.

[17] Petroc Taylor. *Number of Internet of Things (IoT) connected devices worldwide from 2019 to 2034, by vertical*. Statista Research Report. 2025. URL: https://www.statista.com/statistics/1194682/iot-connected-devices-vertically/.

[18] Lawrence C. Washington. *Elliptic Curves: Number Theory and Cryptography*. 2nd. Boca Raton, FL, USA: Chapman és Hall/CRC, 2008. URL: https://people.cs.nycu.edu.tw/~rjchen/ECC2012S/Elliptic%20Curves%20Number%20Theory%20And%20Cryptography%202n.pdf.